# JPA Mapping in Spring Boot

## Complete Interview Preparation Notes

*Based on: Student Management System Project*

**What's Inside:**

Section 01 — What is JPA, Hibernate, Spring Data JPA
Section 02 — @Entity, @Table, @Column, @Id, @GeneratedValue
Section 03 — @Embeddable and @Embedded
Section 04 — @OneToOne Relationship
Section 05 — @OneToMany and @ManyToOne
Section 06 — @ManyToMany Relationship
Section 07 — FetchType, CascadeType, N+1 Problem
Section 08 — Inheritance Mapping (SINGLE_TABLE, JOINED, TABLE_PER_CLASS)
Section 09 — Spring Data JPA Repository & Pagination
Section 10 — @Query, JPQL, Native SQL, DTO Projections
Section 11 — @Transactional, Propagation, Isolation
Section 12 — @NamedQuery, Locking, @MappedSuperclass, Auditing
Section 13 — Entity Lifecycle, Callbacks, Dirty Checking
Section 14 — Top 20 Q&A + Complete Annotations Cheat Sheet

# SECTION 01 What is JPA? — The Foundation

## 1.1 JPA in Simple Words

JPA stands for Java Persistence API. The word 'persistence' means 'saving data permanently'. So JPA is a way to save Java objects permanently into a database without writing SQL manually.

Think of it like this: You have a Java class Student. JPA will automatically create a student table in the database and save your Student objects as rows in that table. You never need to write INSERT INTO student... yourself.

> 💡 **Remember**
> JPA is only a SPECIFICATION (set of rules). It does not run anything by itself. Hibernate is the actual engine that implements these rules. Think of JPA as a recipe and Hibernate as the cook.

## 1.2 Without JPA vs With JPA

| Without JPA (Old Way) | With JPA (Modern Way) |
|---|---|
| Write SQL queries manually for every operation | JPA auto-generates SQL for you |
| Open and close DB connections yourself | Spring manages connection pooling |
| Convert DB rows to Java objects manually (ResultSet) | JPA maps rows to objects automatically |
| Update SQL if table changes — lots of code changes | Change Java class — JPA adapts automatically |
| Risk of SQL injection if not careful | JPA uses prepared statements by default |

## 1.3 JPA vs Hibernate vs Spring Data JPA — What is the difference?

This is one of the most common interview questions. Learn this clearly.

| Name | Type | What it does | Analogy |
|---|---|---|---|
| JPA | Specification / Rules | Defines how Java classes should map to DB tables. Just a set of interfaces and annotations. | A blueprint for a house |
| Hibernate | Implementation of JPA | The actual code that follows JPA rules and communicates with the database. Most popular JPA provider. | The construction company that builds from the blueprint |
| Spring Data JPA | Abstraction on top of JPA/Hibernate | Gives you ready-made CRUD methods (save, findAll, delete) without writing any SQL or JPQL. | A furnished house — |

| | | | |
|---|---|---|---|
| | | | everything ready to use |
| JDBC | Low-level DB access | Direct database access using SQL. JPA uses JDBC internally under the hood. | Raw bricks and cement |

## 1.4 Student Management System — Our Project

Throughout this entire document, we will use a Student Management System as our reference project. This is a real-world application with the following entities:

- **Student** — has name, email, phone, address, courses, and a library card
- **Course** — has title, credits; a course can have many students
- **Address** — has street, city, state, pincode (embedded inside Student)
- **LibraryCard** — each student gets exactly one library card
- **Department** — a department has many students
- **Exam** — student appears in many exams; each exam has many students

```
// Student Management System — Entity Overview
```

```
// The 6 Entities we will build throughout this document:
Student, Course, Address (Embeddable), LibraryCard, Department, Exam

// Database tables that will be created automatically by JPA:
student          --> Student entity
course           --> Course entity
library_card     --> LibraryCard entity
department       --> Department entity
exam             --> Exam entity
student_course   --> Join table for Student <-> Course (ManyToMany)
```

> 💡 **Interview Tip**
> When the interviewer asks 'Tell me about a project where you used JPA', describe this Student Management System confidently. Mention: entities, relationships, repositories, custom queries. It covers all JPA topics.

**Q: What is JPA and why do we use it in Spring Boot?**

**A:** JPA (Java Persistence API) is a specification that defines how Java objects are mapped to database tables. We use it in Spring Boot because it removes the need to write boilerplate SQL code. Hibernate implements JPA and does the actual DB work. Spring Data JPA on top of that gives us CRUD methods for free. Together, they save a lot of development time and reduce bugs.

**Q: What is the difference between JPA and Hibernate?**

**A:** JPA is just a set of rules (interfaces and annotations) — it cannot run by itself. Hibernate is the actual implementation that follows JPA rules and talks to the database. You can swap

> Hibernate with another JPA provider like EclipseLink, but most developers use Hibernate because it is the most mature and feature-rich.

# SECTION 02 @Entity, @Table, @Column, @Id — Mapping a Class

## 2.1 What is an Entity?

An Entity is a Java class that represents a table in the database. Each field in the class = one column in the table. Each object (instance) of the class = one row in the table.

| Java World | Database World |
|---|---|
| Java Class (Student) | Database Table (student) |
| Field (String name) | Column (name VARCHAR) |
| Object (new Student()) | Row (one record in the table) |
| Field value ("Ravi") | Column value ('Ravi') |

## 2.2 Student Entity — Full Example with Explanation

```
// Student.java — @Entity example

@Entity                                     // 1. Tells JPA: this class is a DB table
@Table(name = "student")                    // 2. Table name in DB = 'student'
public class Student {

    @Id                                     // 3. This field is the Primary Key
    @GeneratedValue(strategy = GenerationType.IDENTITY)   // 4. Auto-increment
(1,2,3...)
    private Long id;

    @Column(name = "full_name",             // 5. Column name = 'full_name'
            nullable = false,                // 6. Cannot be NULL in DB
            length = 100)                   // 7. VARCHAR(100)
    private String name;

    @Column(unique = true,                  // 8. No two students can have same email
            nullable = false)
    private String email;

    @Column(name = "phone", length = 15)
    private String phone;

    @Column(name = "enrollment_year")
    private int enrollmentYear;

    @Column(name = "is_active",
            columnDefinition = "BOOLEAN DEFAULT TRUE")  // 9. Custom SQL
definition
    private boolean active;
```

```
    @Transient                          // 10. NOT stored in DB (calculated
field)
    private String displayLabel;         // e.g., "Ravi (2023)" — only in memory

    // Getters, Setters, Constructors...
}
```

### 2.3 Every Annotation Explained

| Annotation | Where to use | What it does | If you skip it |
|---|---|---|---|
| @Entity | On the class | Marks this Java class as a JPA-managed DB table | JPA ignores this class completely |
| @Table(name=) | On the class | Sets custom table name in the database | JPA uses the class name as table name (lowercase) |
| @Id | On one field | Marks this field as the primary key (PK) | Error — every entity MUST have @Id |
| @GeneratedValue | On the @Id field | Auto-generates PK value (1, 2, 3...) | You must manually set the ID yourself |
| @Column(name=) | On any field | Maps field to a specific column name in DB | JPA uses field name as column name |
| @Column(nullable=false) | On any field | Adds NOT NULL constraint in DB | Column allows NULL values |
| @Column(unique=true) | On any field | Adds UNIQUE constraint in DB | Duplicate values allowed |
| @Column(length=) | On String fields | Sets VARCHAR length (default 255) | VARCHAR(255) by default |
| @Transient | On any field | Field is NOT saved in DB at all | Field would be mapped to a column |

## 2.4 @GeneratedValue Strategies — Explained

This tells JPA HOW to generate the primary key value automatically. There are 4 strategies:

| Strategy | How it works | Best used for | Example ID generated |
|---|---|---|---|
| IDENTITY | DB auto-increments (like MySQL AUTO_INCREMENT) | MySQL, H2, PostgreSQL | 1, 2, 3, 4 ... |
| SEQUENCE | DB uses a sequence object to generate IDs | PostgreSQL, Oracle | 1, 51, 101 ... (by 50) |
| TABLE | JPA creates a separate table to track IDs | Any database (portable) | Slow — avoid in production |
| AUTO (default) | JPA picks best strategy for your DB automatically | Quick start / prototyping | Depends on DB |

```
// @GeneratedValue strategies
```

```
// Most common — use IDENTITY for MySQL/H2
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;

// For Oracle/PostgreSQL — using SEQUENCE
@Id
@GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "student_seq")
@SequenceGenerator(name = "student_seq", sequenceName = "STUDENT_SEQ",
allocationSize = 1)
private Long id;
```

## 2.5 application.properties — JPA Configuration

```
// application.properties for Student Management System
```

```
# Database connection
spring.datasource.url=jdbc:mysql://localhost:3306/student_db
spring.datasource.username=root
spring.datasource.password=yourpassword
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# JPA / Hibernate settings
spring.jpa.hibernate.ddl-auto=update       # create / update / validate /
create-drop / none
spring.jpa.show-sql=true                   # prints SQL in console — helpful for
debugging
spring.jpa.properties.hibernate.format_sql=true   # pretty-prints the SQL
spring.jpa.database-platform=org.hibernate.dialect.MySQL8Dialect
```

| ddl-auto value | What it does | Use in |
|---|---|---|
| create | Drops and creates tables every time app starts. All data is LOST. | Initial development only |
| update | Updates table if entity changes. Data is NOT lost. | Development |
| validate | Only checks if DB matches entities. No changes made. Throws error if mismatch. | Production |
| create-drop | Creates tables on start, drops on stop. | Testing / unit tests |
| none | Does nothing. You manage DB manually. | Production (recommended) |

> 💡 **Interview Tip**
> In interviews, always mention: 'In production, we should use ddl-auto=validate or none and use Flyway or Liquibase for database migrations. Using update in production is risky.'

**Q: What happens if you do not put @Column annotation on a field?**

**A:** JPA will still map that field to a column automatically. The column name will be the same as the field name. All default settings apply: nullable=true, no unique constraint, VARCHAR(255) for Strings. You only need @Column when you want to customize something.

**Q: What is @Transient annotation?**

**A:** @Transient tells JPA to completely ignore this field — it will NOT be stored in the database. Use it for calculated or temporary fields that are only needed in memory. Example: a 'displayLabel' field that combines name and year, used only for UI display.

# SECTION 03 @Embeddable & @Embedded — Value Objects

## 3.1 What is @Embeddable?

Sometimes you have a group of related fields (like Address: street, city, state, pincode) that belong together but do NOT need their own table. You want them stored inside the Student table itself.

This is what @Embeddable is for. The Address class is not an entity (no @Entity, no @Id). It is just a group of fields that get embedded inside the Student table.

| @Embeddable (Address) | @Entity (Student) |
|---|---|
| No @Entity annotation | Has @Entity annotation |
| No @Id (no primary key) | Has @Id (primary key) |
| No separate table created | Creates its own table |
| Fields stored inside the parent table | Is the main table |
| Cannot exist independently | Can exist by itself |

## 3.2 Student Management System — Address Example

```
// Address.java and Student.java
```

```
// Address.java — @Embeddable (NOT an Entity, NO separate table)
@Embeddable
public class Address {

    @Column(name = "street", length = 200)
    private String street;

    @Column(name = "city", length = 100)
    private String city;

    @Column(name = "state", length = 100)
    private String state;

    @Column(name = "pincode", length = 10)
    private String pincode;

    // Constructors, Getters, Setters
}

// Student.java — @Embedded (Address fields go into student table)
@Entity
@Table(name = "student")
public class Student {
```

```
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @Embedded                          // Include Address fields in this table
    private Address address;
}
```

### What the student table looks like in DB:

| Column in student table | Comes from | Java type |
|---|---|---|
| id | Student.id | Long (PRIMARY KEY) |
| name | Student.name | String |
| street | Address.street (embedded) | String |
| city | Address.city (embedded) | String |
| state | Address.state (embedded) | String |
| pincode | Address.pincode (embedded) | String |

Notice: There is NO separate 'address' table. All address columns go directly into the 'student' table.

## 3.3 @AttributeOverride — Reusing @Embeddable in Multiple Entities

What if both Student and Teacher need an Address? You can reuse the same Address class, but override column names to avoid conflicts.

```
  // Teacher.java — @AttributeOverride
```

```
// Teacher.java — reusing Address but with different column names
@Entity
@Table(name = "teacher")
public class Teacher {

    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;

    @Embedded
    @AttributeOverrides({
        @AttributeOverride(name = "street",  column = @Column(name = "t_street")),
        @AttributeOverride(name = "city",    column = @Column(name = "t_city")),
        @AttributeOverride(name = "state",   column = @Column(name = "t_state")),
        @AttributeOverride(name = "pincode", column = @Column(name = "t_pincode"))
    })
    private Address address;
}
```

💡 **Interview Tip**

@Embeddable is great for value objects: Address, PhoneNumber, MoneyAmount, DateRange. These are objects that don't make sense alone — they always belong to a parent entity. Mentioning this in an interview shows senior-level thinking.

**Q: What is the difference between @Embedded and @Entity?**

**A:** @Entity creates its own separate table with a primary key. @Embeddable does NOT create a table — its fields are stored directly inside the parent entity's table. Use @Embeddable when the fields logically belong together but don't need an independent identity (no ID needed). Example: Address fields embedded inside Student table.

**Q: Can the same @Embeddable class be used in multiple entities?**

**A:** Yes! You can use the same @Embeddable class in multiple entities. If column names conflict, use @AttributeOverride to rename the columns in each entity. For example, both Student and Teacher can have an Address, but Teacher uses t_street, t_city etc. column names.

# SECTION 04 @OneToOne — One to One Relationship

## 4.1 What is @OneToOne?

@OneToOne means: One record in Table A has exactly one matching record in Table B. And that record in Table B belongs to only that one record in Table A.

Real-life example from Student Management System: Each Student gets exactly one Library Card. One Library Card belongs to only one Student.

## 4.2 Full Code Example — Student and LibraryCard

```
// Student.java and LibraryCard.java — @OneToOne

// LibraryCard.java
@Entity
@Table(name = "library_card")
public class LibraryCard {

    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "card_number", unique = true, nullable = false)
    private String cardNumber;         // e.g. "LIB-2023-001"

    @Column(name = "issue_date")
    private LocalDate issueDate;

    @Column(name = "expiry_date")
    private LocalDate expiryDate;

    @Column(name = "is_active")
    private boolean active = true;

    // Getters, Setters...
}

// Student.java — owns the relationship (has the foreign key)
@Entity
@Table(name = "student")
public class Student {

    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @OneToOne(cascade = CascadeType.ALL,     // Save/delete card with student
              fetch = FetchType.LAZY,         // Load card only when accessed
              optional = false)               // Student MUST have a card
    @JoinColumn(name = "library_card_id",    // FK column in student table
```

```
                 nullable = false)
    private LibraryCard libraryCard;
}
```

### What the database tables look like:

| student table | library_card table |
|---|---|
| id (PK) | id (PK) |
| name | card_number |
| library_card_id (FK) → references library_card.id | issue_date |
| | expiry_date |
| | is_active |

The student table has a column library_card_id which is a foreign key pointing to library_card.id. This is how the relationship is stored in the database.

## 4.3 Bidirectional @OneToOne

In the example above, you can access libraryCard from Student. But what if you want to also access the student FROM libraryCard? This is called bidirectional mapping.

```
  // LibraryCard.java — bidirectional @OneToOne
```

```
// LibraryCard.java — add the reverse side
@Entity
@Table(name = "library_card")
public class LibraryCard {

    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String cardNumber;

    @OneToOne(mappedBy = "libraryCard")  // 'libraryCard' = field name in Student
    private Student student;              // Now you can do card.getStudent()

    // Getters, Setters...
}
```

> 💡 **Remember**
> mappedBy = 'libraryCard' means: This side (LibraryCard) does NOT own the foreign key. The foreign key is in the Student table. mappedBy always points to the FIELD NAME in the other class that has @JoinColumn.

## 4.4 How to Save and Retrieve — Service Layer

```
// StudentService.java — save @OneToOne
```

```java
// StudentService.java
@Service
public class StudentService {

    @Autowired
    private StudentRepository studentRepo;

    public Student createStudentWithCard(String name, String cardNumber) {

        // Step 1: Create the LibraryCard object
        LibraryCard card = new LibraryCard();
        card.setCardNumber(cardNumber);
        card.setIssueDate(LocalDate.now());
        card.setExpiryDate(LocalDate.now().plusYears(3));

        // Step 2: Create Student and attach the card
        Student student = new Student();
        student.setName(name);
        student.setLibraryCard(card);

        // Step 3: Save — CascadeType.ALL saves the card automatically!
        return studentRepo.save(student);
        // JPA runs: INSERT INTO library_card ... then INSERT INTO student ...
    }
}
```

## 4.5 @OneToOne — All Options Explained

| Option | Values | What it means |
|---|---|---|
| cascade | ALL, PERSIST, MERGE, REMOVE, REFRESH | Which operations on Student also apply to LibraryCard |
| fetch | LAZY, EAGER | LAZY=load card only when needed. EAGER=load card always with student |
| optional | true (default), false | false means Student MUST have a LibraryCard (NOT NULL) |
| mappedBy | field name | Marks this side as non-owner. The other side has the FK. |

> 💡 **Interview Tip**
> Always use fetch=FetchType.LAZY on @OneToOne unless you ALWAYS need the related object. EAGER loading can cause performance issues because it always joins the tables even when you don't need the related data.

> **Q: What is the difference between unidirectional and bidirectional @OneToOne?**

**A:** Unidirectional means only one side knows about the relationship (Student has libraryCard field but LibraryCard does not have a student field). Bidirectional means both sides know about each other. The side with @JoinColumn is the OWNER (has the foreign key). The other side uses mappedBy to say 'the FK is over there'.

**Q: What does optional=false do in @OneToOne?**

**A:** optional=false means this relationship is mandatory. The student MUST have a LibraryCard. JPA adds a NOT NULL constraint on the foreign key column (library_card_id). If you try to save a student without a library card, JPA throws a constraint violation exception.

# SECTION 05 @OneToMany & @ManyToOne — Most Used Relationship

## 5.1 What is @OneToMany / @ManyToOne?

This is the most commonly used relationship in real projects. One parent has a LIST of children. Each child belongs to one parent.

Real-life example: One Department has MANY Students. Each Student belongs to ONE Department.

| Side | Annotation | Owns FK? | Example |
|---|---|---|---|
| Department (One) | @OneToMany | NO — does not have FK | Department has List<Student> |
| Student (Many) | @ManyToOne | YES — has dept_id column | Student has Department dept |

## 5.2 Full Code Example — Department and Student

```
// Department.java — @OneToMany
```

```
// Department.java — the 'One' side
@Entity
@Table(name = "department")
public class Department {

    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String name;         // e.g., "Computer Science"

    private String hodName;       // Head of Department

    @OneToMany(
        mappedBy = "department",         // 'department' field name in Student
        cascade = CascadeType.ALL,        // Save/delete students with department
        fetch = FetchType.LAZY,           // Don't load students until accessed
        orphanRemoval = true              // If student removed from list, delete
from DB
    )
    private List<Student> students = new ArrayList<>();

    // Helper method — always add both sides in bidirectional
    public void addStudent(Student student) {
        students.add(student);
        student.setDepartment(this);
    }
```

```
    public void removeStudent(Student student) {
        students.remove(student);
        student.setDepartment(null);
    }

    // Getters, Setters...
}
```

```
// Student.java — @ManyToOne
```

```
// Student.java — the 'Many' side (owns the foreign key)
@Entity
@Table(name = "student")
public class Student {

    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @ManyToOne(fetch = FetchType.LAZY)          // Many students -> one department
    @JoinColumn(name = "dept_id",               // FK column name in student table
                nullable = false)               // Student must have a dept
    private Department department;

    // Getters, Setters...
}
```

### Database Tables:

| student table | department table |
|---|---|
| id (PK) | id (PK) |
| name | name |
| dept_id (FK) --> department.id | hod_name |
| | (No student columns here!) |

The student table has a dept_id foreign key. The department table has NO student columns. JPA manages this entirely through the @ManyToOne side.

## 5.3 Using the Relationship — Service Layer

```
// StudentService.java — using @OneToMany/@ManyToOne
```

```
// StudentService.java
@Service
@Transactional
public class StudentService {

    @Autowired StudentRepository studentRepo;
```

```
    @Autowired DepartmentRepository deptRepo;

    // Enroll student in a department
    public Student enrollStudent(String studentName, Long deptId) {
        Department dept = deptRepo.findById(deptId)
                                .orElseThrow(() -> new RuntimeException("Dept not
found"));

        Student student = new Student();
        student.setName(studentName);
        student.setDepartment(dept);      // set the FK side

        return studentRepo.save(student);
        // SQL: INSERT INTO student (name, dept_id) VALUES ('Ravi', 1)
    }

    // Get all students of a department
    public List<Student> getStudentsByDept(Long deptId) {
        Department dept = deptRepo.findById(deptId)
                                .orElseThrow(() -> new RuntimeException("Dept not
found"));
        return dept.getStudents();
        // SQL: SELECT * FROM student WHERE dept_id = 1
    }
}
```

## 5.4 orphanRemoval — Very Important

orphanRemoval = true means: if you REMOVE a student from the department's list, JPA will automatically DELETE that student from the database.

```
  // orphanRemoval example
```

```
// With orphanRemoval = true
Department dept = deptRepo.findById(1L).get();
Student student = dept.getStudents().get(0);

dept.getStudents().remove(student);    // Remove from list
deptRepo.save(dept);
// JPA runs: DELETE FROM student WHERE id = ? (student is deleted from DB!)

// Without orphanRemoval = true, the student stays in DB
// but dept_id becomes NULL (just unlinked, not deleted)
```

> 💡 **Interview Tip**
> The most important rule: ALWAYS set the FK on the @ManyToOne side (Student). Just setting the list on the @OneToMany side (Department) is NOT enough to create the foreign key in DB. The @ManyToOne side owns the relationship.

> **Q: What is mappedBy in @OneToMany?**

**A:** mappedBy tells JPA that this side (Department) does NOT own the foreign key. The field name you put in mappedBy (e.g., 'department') is the field in the other class (Student) that has @JoinColumn. JPA looks at that field to find the FK column. Without mappedBy, JPA creates an extra join table — which you don't want.

**Q: What is orphanRemoval?**

**A:** orphanRemoval=true means: if a child entity (Student) is removed from the parent's collection (Department.students list), and that child has no other parent, JPA will automatically DELETE it from the database. This is different from cascade=REMOVE which deletes children when the parent is deleted.

# SECTION 06 @ManyToMany — Many to Many Relationship

## 6.1 What is @ManyToMany?

Many records in Table A relate to many records in Table B. And each record in Table B also relates to many in Table A.

Real-life example: A Student can enroll in MANY Courses. A Course can have MANY Students enrolled in it.

JPA handles this by creating a third table (called a JOIN TABLE or BRIDGE TABLE) automatically. This table stores pairs of IDs — (student_id, course_id).

| Table | Columns |
|---|---|
| student | id, name, email, ... |
| course | id, title, credits, ... |
| student_course (join table) | student_id (FK), course_id (FK) |

## 6.2 Full Code Example — Student and Course

```
// Course.java — @ManyToMany
```

```
// Course.java
@Entity
@Table(name = "course")
public class Course {

    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String title;         // e.g., "Java Programming"

    private int credits;          // e.g., 4

    private String instructor;

    @ManyToMany(mappedBy = "courses",  // 'courses' = field in Student class
                fetch = FetchType.LAZY)
    private List<Student> students = new ArrayList<>();

    // Getters, Setters...
}
```

```
// Student.java — @ManyToMany with @JoinTable
```

```java
// Student.java — OWNER side (has @JoinTable)
@Entity
@Table(name = "student")
public class Student {

    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @ManyToMany(cascade = {CascadeType.PERSIST, CascadeType.MERGE},
                fetch = FetchType.LAZY)
    @JoinTable(
        name = "student_course",                        // Name of join table
        joinColumns = @JoinColumn(name = "student_id"),  // FK to student
        inverseJoinColumns = @JoinColumn(name = "course_id")  // FK to course
    )
    private List<Course> courses = new ArrayList<>();

    // Helper methods for bidirectional sync
    public void addCourse(Course course) {
        courses.add(course);
        course.getStudents().add(this);
    }

    public void removeCourse(Course course) {
        courses.remove(course);
        course.getStudents().remove(this);
    }

    // Getters, Setters...
}
```

> 💡 **Remember**
> Do NOT use cascade=REMOVE or cascade=ALL on @ManyToMany. If you delete a student, you do NOT want to delete all the courses! Only use PERSIST and MERGE. The join table rows will be removed automatically.

## 6.3 Using @ManyToMany in Service Layer

```java
// CourseEnrollmentService.java
```

```java
// CourseEnrollmentService.java
@Service
@Transactional
public class CourseEnrollmentService {

    @Autowired StudentRepository studentRepo;
    @Autowired CourseRepository courseRepo;

    // Enroll a student in a course
    public void enrollStudentInCourse(Long studentId, Long courseId) {
        Student student = studentRepo.findById(studentId)
```

```
                            .orElseThrow(() -> new RuntimeException("Student not
found"));
        Course course = courseRepo.findById(courseId)
                           .orElseThrow(() -> new RuntimeException("Course not
found"));

        student.addCourse(course);    // updates both sides
        studentRepo.save(student);
        // JPA runs: INSERT INTO student_course (student_id, course_id) VALUES (1,
3)
    }

    // Remove student from a course
    public void unenrollStudent(Long studentId, Long courseId) {
        Student student = studentRepo.findById(studentId).get();
        Course course = courseRepo.findById(courseId).get();
        student.removeCourse(course);
        studentRepo.save(student);
        // JPA runs: DELETE FROM student_course WHERE student_id=1 AND course_id=3
    }
}
```

## 6.4 @ManyToMany with Extra Columns — @Embeddable Join Table

What if the join table needs extra data? For example: enrollment date, grade, etc. In that case, create a separate Entity for the join table.

```
  // Enrollment.java — join entity with extra columns
```

```
// Enrollment.java — Entity for join table with extra data
@Entity
@Table(name = "enrollment")
public class Enrollment {

    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne
    @JoinColumn(name = "student_id")
    private Student student;

    @ManyToOne
    @JoinColumn(name = "course_id")
    private Course course;

    // Extra data on the relationship!
    private LocalDate enrollmentDate;
    private String grade;             // 'A', 'B', 'C'
    private boolean completed;
}
```

💡 **Interview Tip**

When the join table needs extra columns (like enrollment date, grade), convert it from a @ManyToMany to two @ManyToOne relationships pointing to a new Entity. This is a very common real-world pattern.

**Q: Why should we not use cascade=ALL or cascade=REMOVE on @ManyToMany?**

**A:** Because cascade=REMOVE would delete all related entities when you delete one side. For example, if you delete a Student with cascade=ALL on courses, JPA would try to delete all Courses that student was enrolled in — which would affect all other students enrolled in those same courses. Only use cascade=PERSIST and cascade=MERGE for @ManyToMany.

**Q: What is @JoinTable annotation?**

**A:** @JoinTable defines the join/bridge table for a @ManyToMany relationship. You specify: the name of the join table, the FK column pointing to the owning entity (joinColumns), and the FK column pointing to the other entity (inverseJoinColumns). If you don't specify @JoinTable, JPA auto-generates the table and column names.

# SECTION 07 FetchType, CascadeType & N+1 Problem

## 7.1 FetchType — LAZY vs EAGER

FetchType controls WHEN JPA loads related data from the database. This is very important for performance.

| FetchType | When data is loaded | Default for | Performance |
|---|---|---|---|
| LAZY | Only when you CALL the getter (.getCourses()) | @OneToMany, @ManyToMany | Better — loads only when needed |
| EAGER | IMMEDIATELY when parent is loaded (always JOIN) | @ManyToOne, @OneToOne | Can be slow if data is large |

```
// LAZY vs EAGER loading
```

```
// LAZY example — courses NOT loaded until you call getCourses()
@ManyToMany(fetch = FetchType.LAZY)
private List<Course> courses;

// In service:
Student s = studentRepo.findById(1L).get();
// At this point: SELECT * FROM student WHERE id=1 (NO join yet)

s.getCourses();   // NOW: SELECT * FROM student_course JOIN course WHERE
student_id=1

// ————————————————————————————————————————————
// EAGER example — department loaded ALWAYS with student
@ManyToOne(fetch = FetchType.EAGER)
private Department department;

// In service:
Student s = studentRepo.findById(1L).get();
// SQL: SELECT s.*, d.* FROM student s JOIN department d ON s.dept_id = d.id WHERE
s.id=1
// Department is loaded even if you never use it!
```

## 7.2 The N+1 Problem — Most Important Interview Topic

N+1 is a very common performance problem. Every interviewer who works with JPA knows about this. You MUST understand it.

The problem: You load N students, and then JPA fires 1 extra SQL query for each student to get their department. Total = N+1 queries. If N=100, that is 101 queries!

```
// N+1 Problem and Solutions
```

```
// PROBLEM — N+1 queries with LAZY loading
List<Student> students = studentRepo.findAll();
```

```
// SQL 1: SELECT * FROM student   (gets 100 students)

for (Student s : students) {
    System.out.println(s.getDepartment().getName());
    // SQL 2 to 101: SELECT * FROM department WHERE id=? (1 query PER student!)
}
// Total: 101 SQL queries! This is the N+1 problem.

// ═══════════════════════════════════════════════
// SOLUTION 1 — JOIN FETCH in JPQL (best solution)
@Query("SELECT s FROM Student s JOIN FETCH s.department")
List<Student> findAllWithDepartment();
// SQL: SELECT s.*, d.* FROM student s JOIN department d ON s.dept_id=d.id
// Only 1 query! Problem solved.

// SOLUTION 2 — @EntityGraph (Spring Data JPA way)
@EntityGraph(attributePaths = {"department"})
@Query("SELECT s FROM Student s")
List<Student> findAllWithDeptGraph();
```

## 7.3 CascadeType — All Types Explained

Cascade means: when you perform an operation on the parent entity, automatically perform the same operation on the child entity.

| CascadeType | What it does | Real Example in SMS |
|---|---|---|
| PERSIST | When you save (persist) parent, child is also saved | Save Student -> LibraryCard is also saved |
| MERGE | When you update parent, child is also updated | Update Student -> LibraryCard is also updated |
| REMOVE | When you delete parent, child is also deleted | Delete Student -> LibraryCard is also deleted |
| REFRESH | When you refresh parent from DB, child is also refreshed | Refresh Student -> LibraryCard reloaded from DB |
| DETACH | When parent is detached from session, child also detaches | Rarely used directly |
| ALL | All of the above (PERSIST+MERGE+REMOVE+REFRESH+DETACH) | Most commonly used shortcut |

```
// CascadeType usage guide
```

```
// When to use which CascadeType:

// @OneToOne — use ALL (lifecycle of card = lifecycle of student)
@OneToOne(cascade = CascadeType.ALL)
private LibraryCard libraryCard;

// @OneToMany — use ALL with orphanRemoval
```

```
@OneToMany(mappedBy = "department", cascade = CascadeType.ALL, orphanRemoval =
true)
private List<Student> students;

// @ManyToMany — use only PERSIST and MERGE (never REMOVE!)
@ManyToMany(cascade = {CascadeType.PERSIST, CascadeType.MERGE})
private List<Course> courses;
```

💡 **Interview Tip**

The N+1 problem is one of the most important JPA interview topics. Always mention: 'We can solve N+1 by using JOIN FETCH in JPQL, or @EntityGraph in Spring Data JPA, or by using FetchType.EAGER (but only when always needed).' This answer shows you understand performance.

**Q: What is the N+1 problem in JPA?**

**A:** N+1 happens when you load a list of N entities and JPA fires 1 extra SQL query for each entity to load its lazy-loaded associations. Total = N+1 queries. For example, loading 50 students and then accessing each student's department triggers 50 extra queries = 51 total. Fix it using JOIN FETCH in JPQL or @EntityGraph.

**Q: What is the default FetchType for @OneToMany and @ManyToOne?**

**A:** @OneToMany and @ManyToMany default to FetchType.LAZY. @ManyToOne and @OneToOne default to FetchType.EAGER. Always prefer LAZY for collections to avoid loading large datasets unnecessarily. Only use EAGER when you are absolutely sure you always need that data.

# SECTION 08 Inheritance Mapping — SINGLE_TABLE, JOINED, TABLE_PER_CLASS

## 8.1 Why Inheritance Mapping?

In object-oriented programming, we use inheritance (parent and child classes). But in databases, there is no inheritance. JPA bridges this gap with Inheritance Mapping.

Example: In our Student Management System, imagine we have a Person parent class. Both Student and Teacher extend Person. How do we store this in a database?

## 8.2 Strategy 1 — SINGLE_TABLE (Most Common)

All classes (Person, Student, Teacher) are stored in ONE table. A special column called 'discriminator' tells which class each row belongs to.

```
// SINGLE_TABLE inheritance strategy
```

```
// Person.java — parent class
@Entity
@Table(name = "person")
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name = "person_type",          // discriminator column
                     discriminatorType = DiscriminatorType.STRING)
public abstract class Person {

    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    private String email;
}

// Student.java — child class
@Entity
@DiscriminatorValue("STUDENT")       // stored in person_type column
public class Student extends Person {
    private String rollNumber;
    private int enrollmentYear;
}

// Teacher.java — child class
@Entity
@DiscriminatorValue("TEACHER")       // stored in person_type column
public class Teacher extends Person {
    private String employeeCode;
    private String subject;
}
```

The single 'person' table looks like this:

| id | person_type | name | email | roll_number | enrollment_year | employee_code | subject |
|---|---|---|---|---|---|---|---|
| 1 | STUDENT | Ravi | ravi@... | CS2023001 | 2023 | NULL | NULL |
| 2 | TEACHER | Dr. Mehta | mehta@... | NULL | NULL | TCH001 | Java |

> 💡 **Remember**
> SINGLE_TABLE is the default if you don't specify a strategy. NULL values appear in columns that don't belong to a particular type. Fast queries (no JOINs needed) but wastes space with NULLs.

## 8.3 Strategy 2 — JOINED (Normalized)

Parent table has common columns. Each child table has only its OWN extra columns. They are joined by primary key.

```
// JOINED inheritance strategy
```

```
@Entity
@Table(name = "person")
@Inheritance(strategy = InheritanceType.JOINED)
public abstract class Person {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private String email;
}

@Entity
@Table(name = "student")   // separate table with ONLY student-specific columns
public class Student extends Person {
    private String rollNumber;         // student-specific
    private int enrollmentYear;        // student-specific
}

// person table: id, name, email
// student table: id (FK to person.id), roll_number, enrollment_year
// teacher table: id (FK to person.id), employee_code, subject
```

## 8.4 Strategy 3 — TABLE_PER_CLASS

Each class gets its OWN COMPLETE table. All parent columns are REPEATED in each child table. No join table needed.

```
// TABLE_PER_CLASS inheritance strategy
```

```
@Entity
```

```
@Inheritance(strategy = InheritanceType.TABLE_PER_CLASS)
public abstract class Person {
    @Id @GeneratedValue(strategy = GenerationType.TABLE)   // must use TABLE or
SEQUENCE
    private Long id;
    private String name;
    private String email;
}

// student table: id, name, email, roll_number, enrollment_year  (ALL columns)
// teacher table: id, name, email, employee_code, subject          (ALL columns)
```

| Strategy | Tables created | Query speed | NULL values | Best for |
|---|---|---|---|---|
| SINGLE_TABLE | 1 table | Fastest (no JOIN) | Yes — child-specific columns are NULL | Simple hierarchy, few child-specific fields |
| JOINED | 1 per class | Slower (JOINs needed) | No NULLs | Large hierarchy, many child-specific fields, normalized data |
| TABLE_PER_CLASS | 1 per class | Medium | No NULLs | Rarely used — columns are repeated |

💡 **Interview Tip**
In interviews, most answers should be SINGLE_TABLE for simple cases. But if asked about normalization or when child classes have many unique fields, answer JOINED. TABLE_PER_CLASS is rarely recommended.

**Q: What is a discriminator column?**

**A:** A discriminator column is used in SINGLE_TABLE inheritance to store the type of each row. For example, if person_type='STUDENT', JPA knows that row is a Student object. If person_type='TEACHER', it maps to a Teacher object. You define it with @DiscriminatorColumn on the parent and @DiscriminatorValue on each child.

**Q: What is the problem with SINGLE_TABLE strategy?**

**A:** In SINGLE_TABLE, all child-specific columns are in one table. If a column belongs to Student but not Teacher, that column will be NULL for Teacher rows. This violates third normal form (3NF) and wastes space. Also, you cannot add NOT NULL constraints on child-specific columns. Use JOINED strategy if data integrity is more important than query speed.

# SECTION 09 Spring Data JPA — Repository & CRUD

## 9.1 Repository Hierarchy — Understanding the Layers

Spring Data JPA has a hierarchy of repository interfaces. Each adds more features than the one above it.

| Interface | Extends | Methods provided | Use when |
|---|---|---|---|
| Repository<T,ID> | nothing | None — just a marker | You want to define all methods yourself |
| CrudRepository<T,ID> | Repository | save, findById, findAll, delete, count, exists | Basic CRUD only |
| PagingAndSortingRepository<T,ID> | CrudRepository | All CRUD + findAll(Pageable), findAll(Sort) | Need pagination and sorting |
| JpaRepository<T,ID> | PagingAndSortingRepository | All above + saveAll, flush, deleteInBatch, getReferenceById | Most common choice — use this always |

## 9.2 StudentRepository — Full Example

```
// StudentRepository.java — derived query methods
```

```
// StudentRepository.java
// JpaRepository<Student, Long> = Entity class, ID type
@Repository
public interface StudentRepository extends JpaRepository<Student, Long> {

    // ── Derived Query Methods (JPA generates SQL from method name) ──

    // SELECT * FROM student WHERE name = ?
    List<Student> findByName(String name);

    // SELECT * FROM student WHERE email = ?
    Optional<Student> findByEmail(String email);

    // SELECT * FROM student WHERE dept_id = ?
    List<Student> findByDepartmentId(Long deptId);

    // SELECT * FROM student WHERE department.name = ?
    List<Student> findByDepartmentName(String deptName);

    // SELECT * FROM student WHERE name LIKE '%?%'
    List<Student> findByNameContainingIgnoreCase(String keyword);

    // SELECT * FROM student WHERE enrollment_year BETWEEN ? AND ?
    List<Student> findByEnrollmentYearBetween(int startYear, int endYear);

    // SELECT COUNT(*) FROM student WHERE dept_id = ?
```

```
    long countByDepartmentId(Long deptId);

    // SELECT * FROM student WHERE is_active = true
    List<Student> findByActiveTrue();

    // DELETE FROM student WHERE name = ?
    void deleteByName(String name);

    // SELECT * FROM student WHERE name = ? AND enrollment_year = ?
    List<Student> findByNameAndEnrollmentYear(String name, int year);

    // SELECT * FROM student WHERE name = ? OR email = ?
    List<Student> findByNameOrEmail(String name, String email);

    // SELECT * FROM student ORDER BY name ASC
    List<Student> findAllByOrderByNameAsc();
}
```

## 9.3 Derived Query Method Keywords — Full List

| Keyword | Method Example | SQL Generated |
|---|---|---|
| findBy | findByName(String n) | WHERE name = ? |
| findAllBy | findAllByActiveTrue() | WHERE is_active = true |
| countBy | countByDepartmentId(Long id) | COUNT(*) WHERE dept_id = ? |
| existsBy | existsByEmail(String email) | EXISTS (WHERE email = ?) |
| deleteBy | deleteByName(String n) | DELETE WHERE name = ? |
| And | findByNameAndEmail(n, e) | WHERE name=? AND email=? |
| Or | findByNameOrEmail(n, e) | WHERE name=? OR email=? |
| Between | findByYearBetween(s, e) | WHERE year BETWEEN ? AND ? |
| LessThan | findByYearLessThan(y) | WHERE year < ? |
| GreaterThan | findByYearGreaterThan(y) | WHERE year > ? |
| Like | findByNameLike(p) | WHERE name LIKE ? |
| Containing | findByNameContaining(k) | WHERE name LIKE '%?%' |
| StartingWith | findByNameStartingWith(p) | WHERE name LIKE '?%' |
| EndingWith | findByNameEndingWith(s) | WHERE name LIKE '%?' |
| IgnoreCase | findByNameIgnoreCase(n) | WHERE UPPER(name)=UPPER(?) |
| OrderBy...Asc | findAllByOrderByNameAsc() | ORDER BY name ASC |
| OrderBy...Desc | findAllByOrderByNameDesc() | ORDER BY name DESC |
| Not | findByNameNot(n) | WHERE name != ? |
| In | findByDeptIdIn(ids) | WHERE dept_id IN (?,?,?) |

| | | |
|---|---|---|
| IsNull | findByEmailIsNull() | WHERE email IS NULL |
| IsNotNull | findByEmailIsNotNull() | WHERE email IS NOT NULL |

## 9.4 Pagination and Sorting

```
// Pagination and Sorting
```

```
// StudentRepository.java — add pagination methods
public interface StudentRepository extends JpaRepository<Student, Long> {

    Page<Student> findByDepartmentId(Long deptId, Pageable pageable);
    Page<Student> findByNameContaining(String keyword, Pageable pageable);
}

// StudentService.java — using Pageable
public Page<Student> getStudentsByDept(Long deptId, int page, int size) {

    // page=0 means first page, size=10 means 10 records per page
    Pageable pageable = PageRequest.of(page, size, Sort.by("name").ascending());

    Page<Student> result = studentRepo.findByDepartmentId(deptId, pageable);

    result.getContent();          // List<Student> — actual data
    result.getTotalElements();    // Total count (e.g., 150)
    result.getTotalPages();       // Total pages (e.g., 15)
    result.getNumber();           // Current page number
    result.isFirst();             // true if first page
    result.isLast();              // true if last page

    return result;
}
```

## 9.5 CRUD Operations — Full Service Example

```
// StudentService.java — full CRUD
```

```
// StudentService.java
@Service
@Transactional
public class StudentService {

    @Autowired
    private StudentRepository studentRepo;

    // CREATE
    public Student createStudent(Student student) {
        return studentRepo.save(student);   // INSERT INTO student...
    }

    // READ by ID
    public Student getById(Long id) {
        return studentRepo.findById(id)
```

```
                .orElseThrow(() -> new RuntimeException("Student not found: " +
id));
    }

    // READ all with pagination
    public Page<Student> getAll(int page, int size) {
        return studentRepo.findAll(PageRequest.of(page, size));
    }

    // UPDATE
    public Student updateStudent(Long id, Student updatedData) {
        Student existing = getById(id);
        existing.setName(updatedData.getName());
        existing.setEmail(updatedData.getEmail());
        return studentRepo.save(existing);  // UPDATE student SET... WHERE id=?
    }

    // DELETE
    public void deleteStudent(Long id) {
        studentRepo.deleteById(id);         // DELETE FROM student WHERE id=?
    }

    // CHECK EXISTENCE
    public boolean exists(Long id) {
        return studentRepo.existsById(id);  // SELECT COUNT(*) WHERE id=?
    }
}
```

💡 **Interview Tip**

JpaRepository<Student, Long>: First parameter is the Entity class (Student). Second parameter is the type of the primary key field. If @Id is Long, use Long. If @Id is Integer, use Integer. This is a very common interview question!

**Q: What is the difference between save() and saveAndFlush()?**

**A:** save() stores the entity in Hibernate's first-level cache (session). The actual SQL may be sent to the DB later, at the end of the transaction. saveAndFlush() immediately sends the SQL to the database without waiting for the transaction to end. Use saveAndFlush() when you need the DB to reflect changes immediately — for example, before calling a stored procedure or for integration tests.

**Q: What is the difference between findById() and getReferenceById()?**

**A:** findById() immediately runs SELECT * FROM table WHERE id=? and returns the actual object (or empty Optional if not found). getReferenceById() returns a proxy object without hitting the DB. The DB is only hit when you access a field on that object. Use getReferenceById() for simple associations where you just need the ID reference without loading all data.

# SECTION 10 @Query — JPQL and Native SQL

## 10.1 What is JPQL?

JPQL stands for Java Persistence Query Language. It looks like SQL but works with Java class and field names instead of table and column names.

This is important: if you rename a column in DB but keep the Java field name, JPQL still works. Native SQL would break.

| Feature | JPQL | Native SQL |
|---|---|---|
| Works with | Java class and field names | Actual DB table and column names |
| Database independent | YES — works on any DB | NO — may differ between MySQL, Oracle, etc. |
| nativeQuery param | false (default) | true |
| Example | FROM Student s WHERE s.name=:n | SELECT * FROM student WHERE name=? |

## 10.2 StudentRepository — @Query Examples

```
// StudentRepository.java — complete @Query examples

@Repository
public interface StudentRepository extends JpaRepository<Student, Long> {

    // ── Basic JPQL ──────────────────────────────────────────

    // Get all students by department name (traverse relationship)
    @Query("SELECT s FROM Student s WHERE s.department.name = :deptName")
    List<Student> findByDeptName(@Param("deptName") String deptName);

    // Find students enrolled in a course (use JOIN)
    @Query("SELECT s FROM Student s JOIN s.courses c WHERE c.title = :title")
    List<Student> findByCourseTitle(@Param("title") String title);

    // Count students per department
    @Query("SELECT d.name, COUNT(s) FROM Student s JOIN s.department d GROUP BY
d.name")
    List<Object[]> countStudentsPerDept();

    // Find students with no courses enrolled
    @Query("SELECT s FROM Student s WHERE s.courses IS EMPTY")
    List<Student> findStudentsWithNoCourses();

    // ── JOIN FETCH (solves N+1 problem) ────────────────────

    @Query("SELECT s FROM Student s JOIN FETCH s.department")
```

```
    List<Student> findAllWithDept();

    @Query("SELECT DISTINCT s FROM Student s LEFT JOIN FETCH s.courses")
    List<Student> findAllWithCourses();   // DISTINCT avoids duplicate rows

    // ── Named Parameters (:name) vs Positional Parameters (?1) ──

    // Named parameter — preferred (clearer)
    @Query("SELECT s FROM Student s WHERE s.name = :name AND s.email = :email")
    Optional<Student> findByNameAndEmail(@Param("name") String name,
                                         @Param("email") String email);

    // Positional parameter — order matters
    @Query("SELECT s FROM Student s WHERE s.name = ?1 AND s.email = ?2")
    Optional<Student> findByNameAndEmailPos(String name, String email);

    // ── Native SQL Query ──────────────────────────────────

    @Query(value = "SELECT * FROM student WHERE dept_id = :deptId ORDER BY name",
           nativeQuery = true)
    List<Student> findByDeptNative(@Param("deptId") Long deptId);

    // ── UPDATE and DELETE queries (@Modifying required) ──────

    @Modifying
    @Transactional
    @Query("UPDATE Student s SET s.active = false WHERE s.enrollmentYear < :year")
    int deactivateOldStudents(@Param("year") int year);

    @Modifying
    @Transactional
    @Query("DELETE FROM Student s WHERE s.active = false")
    int deleteInactiveStudents();
}
```

## 10.3 DTO Projection — Return Only Specific Columns

Sometimes you don't want to load the full Student object. You want only name and email. Use DTO projections.

```
  // DTO Projections
```

```
// Interface-based projection (simplest way)
public interface StudentSummary {
    String getName();
    String getEmail();
    String getDepartmentName();   // from related entity!
}

// In repository:
@Query("SELECT s.name AS name, s.email AS email, s.department.name AS
departmentName
        FROM Student s")
List<StudentSummary> findAllSummary();

// ──────────────────────────────────────────────────
```

```
// Class-based projection (DTO class)
public class StudentDTO {
    private String name;
    private String email;

    public StudentDTO(String name, String email) {   // Constructor must match
        this.name = name;
        this.email = email;
    }
    // Getters...
}

// In repository:
@Query("SELECT new com.sms.dto.StudentDTO(s.name, s.email) FROM Student s")
List<StudentDTO> findStudentDTOs();
```

💡 **Interview Tip**

@Modifying is required for any UPDATE or DELETE @Query. You also need @Transactional (either on the repository method or on the calling service method). Without @Modifying, JPA throws an exception.

**Q: What is the difference between @Query with JPQL and @Query with nativeQuery=true?**

**A:** JPQL (default) uses Java class and field names — it is database-independent and works regardless of the underlying DB. nativeQuery=true uses actual SQL with real table and column names — it is database-specific but gives you full SQL power for complex queries that JPQL cannot express. Prefer JPQL; use native only when JPQL is insufficient.

**Q: What is @Modifying annotation?**

**A:** @Modifying tells Spring Data JPA that this @Query is not a SELECT — it modifies data (UPDATE or DELETE). Without @Modifying, Spring assumes the query is read-only and throws an InvalidDataAccessApiUsageException. Always pair @Modifying with @Transactional.

# SECTION 11 @Transactional — Managing Transactions

## 11.1 What is a Transaction?

A transaction is a group of database operations that must ALL succeed or ALL fail together. If one operation fails in the middle, ALL changes are rolled back (undone).

Real example: Enrolling a student in a course involves 2 steps: (1) Update student record, (2) Insert into student_course join table. If step 2 fails, step 1 should also be undone. This is what @Transactional ensures.

## 11.2 @Transactional in Student Management System

*// EnrollmentService.java — @Transactional*

```
// EnrollmentService.java
@Service
public class EnrollmentService {

    @Autowired StudentRepository studentRepo;
    @Autowired CourseRepository courseRepo;
    @Autowired EmailService emailService;

    // All DB operations inside this method are in ONE transaction
    @Transactional
    public void enrollStudentInCourse(Long studentId, Long courseId) {

        // Step 1: Find student
        Student student = studentRepo.findById(studentId)
                .orElseThrow(() -> new RuntimeException("Student not found"));

        // Step 2: Find course
        Course course = courseRepo.findById(courseId)
                .orElseThrow(() -> new RuntimeException("Course not found"));

        // Step 3: Enroll
        student.addCourse(course);
        studentRepo.save(student);
        // If an exception occurs here, steps 1, 2, 3 are all ROLLED BACK
    }

    // Read-only transaction — tells DB this is just a SELECT (better performance)
    @Transactional(readOnly = true)
    public List<Student> getAllStudents() {
        return studentRepo.findAll();
    }
}
```

## 11.3 @Transactional Attributes

| Attribute | Options | What it does |
|---|---|---|
| readOnly | true / false (default) | Marks transaction as read-only. Better performance for SELECT operations. Hibernate skips dirty-checking. |
| rollbackFor | Exception class (e.g., Exception.class) | By default, rollback only for unchecked (RuntimeException). Use rollbackFor=Exception.class to also rollback for checked exceptions. |
| noRollbackFor | Exception class | Do NOT rollback for this specific exception even if it occurs. |
| propagation | REQUIRED, REQUIRES_NEW, NESTED, etc. | Controls how transaction behaves when calling another @Transactional method. |
| timeout | int (seconds) | Rollback if transaction takes longer than this many seconds. |
| isolation | READ_COMMITTED, REPEATABLE_READ, etc. | Controls how changes are visible across concurrent transactions. |

## 11.4 Transaction Propagation — Important for Interviews

| Propagation | Behavior | Use when |
|---|---|---|
| REQUIRED (default) | Join existing transaction. If none, create new one. | Most common — almost always use this |
| REQUIRES_NEW | Suspend current transaction. Create a completely new one. | Want inner operation to commit independently (e.g., logging) |
| NESTED | Create a savepoint inside current transaction. Can rollback to that savepoint. | Partial rollback scenarios |
| SUPPORTS | Use existing transaction if present. If none, run non-transactionally. | Read operations that can work with or without transaction |
| NOT_SUPPORTED | Suspend current transaction and run without one. | Operations that should not be in a transaction |
| NEVER | Throw exception if called within a transaction. | Operations that must NOT run in a transaction |
| MANDATORY | Throw exception if no existing transaction. | Must always be called from within a transaction |

💡 **Interview Tip**

In interviews: @Transactional should be on the SERVICE layer, not on the repository. Spring Data JPA methods already have @Transactional internally. Adding it on service methods wraps multiple repository calls in ONE transaction — which is the real purpose.

**Q: What happens if an exception is thrown inside a @Transactional method?**

**A:** By default, @Transactional rolls back the transaction for any RuntimeException (unchecked exception). For checked exceptions (e.g., IOException), it does NOT rollback by default. To rollback for checked exceptions too, use @Transactional(rollbackFor = Exception.class). This is a very common interview question!

**Q: What is the difference between REQUIRED and REQUIRES_NEW propagation?**

**A:** REQUIRED joins an existing transaction if one exists, or starts a new one if there is none. REQUIRES_NEW always suspends the current transaction and creates a brand new independent transaction. Use REQUIRES_NEW for operations like audit logging that should commit even if the main transaction rolls back.

# SECTION 12 @NamedQuery, Criteria API & Advanced Features

## 12.1 @NamedQuery — Predefined Queries on Entity

@NamedQuery lets you define a JPQL query directly on the Entity class. It is compiled and validated at application startup — so errors are caught early.

```
// @NamedQuery example
```

```
// Student.java — define named queries on the entity
@Entity
@Table(name = "student")
@NamedQueries({
    @NamedQuery(
        name = "Student.findByDept",
        query = "SELECT s FROM Student s WHERE s.department.name = :deptName"
    ),
    @NamedQuery(
        name = "Student.findActiveStudents",
        query = "SELECT s FROM Student s WHERE s.active = true ORDER BY s.name"
    )
})
public class Student { ... }

// StudentRepository.java — use the named query
public interface StudentRepository extends JpaRepository<Student, Long> {

    // Method name must match the NamedQuery name after the dot
    List<Student> findByDept(@Param("deptName") String deptName);
    List<Student> findActiveStudents();
}
```

## 12.2 @Locking — Optimistic vs Pessimistic

Locking prevents data corruption when multiple users update the same record simultaneously.

```
// Optimistic vs Pessimistic Locking
```

```
// Optimistic Locking — uses a version field to detect conflicts
// Good for low-conflict scenarios (most web apps)
@Entity
public class Student {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Version                    // JPA manages this field automatically
    private int version;        // increments on every UPDATE

    private String name;
```

```
}
// If two users load version=1 and both try to update:
// First update sets version=2. Second update sees version=1 != 2, throws
OptimisticLockException

// ──────────────────────────────────────────────
// Pessimistic Locking — DB locks the row until transaction ends
// Good for high-conflict scenarios (banking, inventory)
@Repository
public interface StudentRepository extends JpaRepository<Student, Long> {

    @Lock(LockModeType.PESSIMISTIC_WRITE)     // DB: SELECT ... FOR UPDATE
    @Query("SELECT s FROM Student s WHERE s.id = :id")
    Optional<Student> findByIdWithLock(@Param("id") Long id);
}
```

## 12.3 @EntityListeners — Auditing with JPA

Auditing automatically tracks who created/updated a record and when. Very useful in real projects.

```
// JPA Auditing with @MappedSuperclass
```

```
// Step 1: Enable JPA Auditing in main class
@SpringBootApplication
@EnableJpaAuditing
public class StudentManagementApp { ... }

// Step 2: Create a base entity with audit fields
@MappedSuperclass                           // Not an Entity itself, but fields
are inherited
@EntityListeners(AuditingEntityListener.class)
public abstract class BaseEntity {

    @CreatedDate
    @Column(name = "created_at", updatable = false)
    private LocalDateTime createdAt;

    @LastModifiedDate
    @Column(name = "updated_at")
    private LocalDateTime updatedAt;

    @CreatedBy
    @Column(name = "created_by", updatable = false)
    private String createdBy;

    @LastModifiedBy
    @Column(name = "updated_by")
    private String updatedBy;
}

// Step 3: Student extends BaseEntity
@Entity
public class Student extends BaseEntity {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
```

```
    private String name;
    // created_at, updated_at, created_by, updated_by are automatically filled!
}
```

## 12.4 @MappedSuperclass vs @Inheritance

| Feature | @MappedSuperclass | @Inheritance |
|---|---|---|
| Creates table? | NO — parent has no table | YES — parent has a table (or shares one) |
| @Id in parent? | YES — inherited by children | YES — in parent's table |
| Query parent type? | NO — cannot query MappedSuperclass | YES — can query parent type |
| Best for | Shared fields (id, createdAt, updatedAt) | Real polymorphism (Student IS-A Person) |

💡 **Interview Tip**
@MappedSuperclass is used for code reuse (DRY principle) — put common fields like id, createdAt in a base class. @Inheritance is used for true object-oriented polymorphism where you need to query by parent type.

**Q: What is Optimistic Locking and when do you use it?**

**A:** Optimistic Locking is used when data conflicts are rare. You add a @Version field to the entity. JPA checks this version on every UPDATE — if another transaction already updated the record, the version won't match and JPA throws OptimisticLockException. The caller can then retry. Best for web applications where users rarely edit the same record simultaneously.

**Q: What is @MappedSuperclass?**

**A:** @MappedSuperclass is a class whose fields are mapped to DB columns but the class itself does NOT become a table. Child entities inherit those fields and they appear in the child's table. It is used to avoid repeating common fields (like id, createdAt, updatedAt) in every entity. Unlike @Inheritance, you cannot query the @MappedSuperclass type directly.

# SECTION 13 Entity Lifecycle & Callbacks

## 13.1 JPA Entity Lifecycle States

A JPA entity goes through different states during its lifetime. Understanding this is important for debugging unexpected behavior.

| State | Description | Example |
|---|---|---|
| Transient (New) | Object created with new keyword. JPA does not know about it. Not in DB. | Student s = new Student(); |
| Managed (Persistent) | Object is tracked by JPA's EntityManager. Any change is automatically synced to DB. | After studentRepo.save(s) or findById() |
| Detached | Was managed, but now the EntityManager session is closed. Changes are NOT synced. | After transaction ends / session closes |
| Removed | Marked for deletion. Will be deleted when transaction commits. | After studentRepo.delete(s) |

```
// Entity lifecycle states
```

```
// Lifecycle state transitions

// 1. TRANSIENT — just created, JPA ignores it
Student s = new Student();
s.setName("Ravi");

// 2. MANAGED — JPA is now tracking this object
studentRepo.save(s);      // Now managed — JPA will sync changes to DB

// 3. Any change to 's' inside the transaction is auto-saved (Dirty Checking)
s.setName("Ravi Kumar");  // JPA detects this change, runs UPDATE automatically

// 4. DETACHED — after transaction ends
// Modifying 's' now will NOT save to DB

// 5. REMOVED
studentRepo.delete(s);    // Marked for deletion, deleted at transaction commit
```

## 13.2 Lifecycle Callback Annotations

| Annotation | When it runs | Common Use Case |
|---|---|---|
| @PrePersist | BEFORE INSERT — just before saving a new entity | Set createdAt = now(), set default values |

| | | |
|---|---|---|
| @PostPersist | AFTER INSERT — after the entity is saved | Send welcome email, log creation |
| @PreUpdate | BEFORE UPDATE — just before updating an existing entity | Set updatedAt = now() |
| @PostUpdate | AFTER UPDATE — after the entity is updated | Send notification, audit log |
| @PreRemove | BEFORE DELETE — just before deleting | Archive data, check business rules |
| @PostRemove | AFTER DELETE — after the entity is deleted | Clean up related files, send notification |
| @PostLoad | AFTER LOADING — after entity is loaded from DB | Decrypt encrypted fields, set calculated fields |

```
// Student.java — lifecycle callbacks
```

```
// Student.java — lifecycle callbacks
@Entity
public class Student {

    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private String email;
    private LocalDateTime createdAt;
    private LocalDateTime updatedAt;
    private String rollNumber;

    @PrePersist
    public void beforeSave() {
        this.createdAt = LocalDateTime.now();     // auto-set creation time
        this.updatedAt = LocalDateTime.now();
        if (this.rollNumber == null) {
            this.rollNumber = "STU-" + System.currentTimeMillis();  // auto roll
number
        }
    }

    @PreUpdate
    public void beforeUpdate() {
        this.updatedAt = LocalDateTime.now();     // auto-update timestamp
    }

    @PostLoad
    public void afterLoad() {
        // e.g., decrypt sensitive fields after loading from DB
    }

    @PreRemove
    public void beforeDelete() {
        System.out.println("Deleting student: " + this.name);
        // Archive to audit table if needed
    }
}
```

## 13.3 Dirty Checking — How JPA Auto-Detects Changes

When an entity is in MANAGED state, JPA takes a 'snapshot' of it when loaded. At the end of the transaction, JPA compares the current state with the snapshot. If anything changed, it automatically runs UPDATE — even if you never called save() again!

```
  // Dirty Checking — auto UPDATE
```

```
// Dirty Checking example
@Transactional
public void updateStudentName(Long id, String newName) {

    Student student = studentRepo.findById(id).get();
    // JPA snapshot: {id=1, name='Ravi', email='ravi@email.com'}

    student.setName(newName);   // Change detected!
    // No need to call studentRepo.save(student) !

    // At end of @Transactional, JPA sees change -> auto runs:
    // UPDATE student SET name='Ravi Kumar' WHERE id=1
}
```

💡 **Interview Tip**

Dirty Checking means you don't always need to call save() to update an entity. If the entity is MANAGED (inside a transaction), just change its fields and JPA will auto-UPDATE. However, it is good practice to call save() explicitly for clarity.

**Q: What is Dirty Checking in JPA?**

**A:** Dirty Checking is a Hibernate feature where it automatically tracks changes to managed entities within a transaction. When you load an entity, Hibernate takes a snapshot. At transaction commit, it compares the current state with the snapshot. If anything changed (entity is 'dirty'), Hibernate automatically issues an UPDATE statement — even without calling save(). This is why you need @Transactional for updates to work automatically.

**Q: What are JPA lifecycle callbacks used for?**

**A:** @PrePersist is commonly used to auto-set createdAt timestamp. @PreUpdate auto-sets updatedAt. @PostLoad can be used to decrypt sensitive fields after loading from DB. @PreRemove can be used to perform validation or archiving before deletion. These callbacks keep business logic inside the entity itself, reducing boilerplate in services.

# SECTION 14 Top Interview Questions & Final Cheat Sheet

## 14.1 Top 20 JPA Interview Questions & Answers

**Q: 1. What is the difference between JPA, Hibernate, and Spring Data JPA?**

**A:** JPA is a specification (set of interfaces/annotations). Hibernate is the most popular implementation of JPA — it does the actual work of talking to the DB. Spring Data JPA is a layer on top of JPA that gives you ready-made repositories with CRUD methods. Think of it as: JPA=rules, Hibernate=worker, Spring Data JPA=smart assistant.

**Q: 2. What annotations are required to create a JPA Entity?**

**A:** Minimum required: @Entity on the class and @Id on the primary key field. @Table and @Column are optional — JPA uses class name as table name and field name as column name by default.

**Q: 3. What is the difference between @OneToOne, @OneToMany, @ManyToOne, @ManyToMany?**

**A:** @OneToOne: one record linked to exactly one other record. @OneToMany: one record linked to a list of records (Department has many Students). @ManyToOne: many records linked to one record (Student belongs to one Department). @ManyToMany: many records linked to many records (Students and Courses). @ManyToMany creates a join table.

**Q: 4. Which side of a relationship owns the foreign key?**

**A:** The side with @JoinColumn annotation owns the foreign key. In @ManyToOne, the 'many' side (Student) has @JoinColumn and owns the FK. The 'one' side (Department) uses mappedBy. In @ManyToMany, the side with @JoinTable is the owner.

**Q: 5. What is FetchType.LAZY vs FetchType.EAGER?**

**A:** LAZY: loads related data only when you access the getter. Efficient — avoids loading unnecessary data. EAGER: loads related data immediately when parent is loaded. Default: @OneToMany/@ManyToMany = LAZY, @ManyToOne/@OneToOne = EAGER. Always use LAZY for collections to avoid performance issues.

**Q: 6. What is the N+1 problem and how do you fix it?**

**A:** N+1 happens when you load N entities and JPA fires 1 extra query per entity for lazy associations. Fix: use JOIN FETCH in JPQL (SELECT s FROM Student s JOIN FETCH s.department) or @EntityGraph. This reduces N+1 queries to 1 query.

**Q: 7. What is CascadeType.ALL vs CascadeType.REMOVE?**

**A:** CascadeType.REMOVE deletes child when parent is deleted. CascadeType.ALL includes REMOVE plus PERSIST, MERGE, REFRESH, DETACH. Use ALL for tight parent-child relationships like Student-LibraryCard. Never use REMOVE or ALL on @ManyToMany — it would delete shared entities.

### Q: 8. What is orphanRemoval=true?

**A:** orphanRemoval=true automatically deletes a child entity from the DB when it is removed from the parent's collection. Different from cascade=REMOVE: cascade=REMOVE deletes all children when parent is deleted. orphanRemoval=true deletes a specific child when it is disassociated from the parent collection.

### Q: 9. What are the JPA Inheritance strategies?

**A:** Three strategies: SINGLE_TABLE (all classes in 1 table, uses discriminator column — fastest), JOINED (parent table + child tables joined by PK — normalized, no NULLs), TABLE_PER_CLASS (each class gets all columns in its own table — rarely used). SINGLE_TABLE is default and most common.

### Q: 10. What is the difference between @Embeddable and @Entity?

**A:** @Entity creates its own table with a primary key. @Embeddable does not create a table — its fields are stored in the parent entity's table. Use @Embeddable for value objects like Address, PhoneNumber that don't have their own identity and don't make sense standalone.

### Q: 11. What is @Transient annotation?

**A:** @Transient tells JPA to NOT map this field to any column. It is used for fields that are only needed in memory — like calculated fields, display labels, or temporary flags that should not be persisted.

### Q: 12. What does @Modifying do?

**A:** @Modifying is required on @Query methods that perform UPDATE or DELETE. Without it, Spring Data JPA assumes the query is a SELECT and throws an exception. Always pair @Modifying with @Transactional.

### Q: 13. What is Dirty Checking?

**A:** Dirty Checking is Hibernate's mechanism to detect changes to managed entities. It takes a snapshot when entity is loaded. At transaction commit, it compares current state to snapshot. If different, it automatically issues UPDATE. This is why you don't always need to call save() explicitly for updates inside @Transactional methods.

### Q: 14. What is the difference between save() and saveAndFlush()?

**A:** save() stores the entity in the persistence context (first-level cache). SQL is sent to DB at end of transaction. saveAndFlush() immediately flushes (sends SQL to DB) without waiting for transaction end. Use saveAndFlush() when you need DB updated immediately, e.g., before calling another method that reads from DB.

**Q: 15. What is @Version annotation?**

**A:** @Version enables Optimistic Locking. JPA adds a version column to the table and increments it on every UPDATE. If two transactions load the same entity and one updates it first, the other sees a version mismatch and throws OptimisticLockException. This prevents lost updates without locking DB rows.

**Q: 16. What is @NamedQuery?**

**A:** @NamedQuery defines a JPQL query on the entity class. It is pre-compiled and validated at application startup, so syntax errors are caught early. Named queries are slightly more efficient than @Query because they are compiled once. Use @NamedQuery for frequently-used queries.

**Q: 17. What is @MappedSuperclass?**

**A:** @MappedSuperclass is a class whose mapped fields are inherited by child entities, but the class itself does not become a DB table. It is used to share common fields (id, createdAt, updatedAt) across multiple entities without creating a parent table. Different from @Inheritance which creates a real table hierarchy.

**Q: 18. What is the ddl-auto property? What should it be in production?**

**A:** ddl-auto controls how Hibernate manages DB schema. Values: create (drop+create on start), update (update schema keeping data), validate (only check, no changes), create-drop (create on start, drop on stop), none (do nothing). In production, use validate or none and manage schema with Flyway or Liquibase.

**Q: 19. What is JPQL? How is it different from SQL?**

**A:** JPQL (Java Persistence Query Language) uses Java class and field names instead of DB table and column names. It is database-independent — same query works on MySQL, Oracle, PostgreSQL. SQL uses actual table/column names and is database-specific. Use JPQL for portability; use native SQL only when JPQL cannot express the query.

**Q: 20. What are JPA lifecycle callbacks?**

**A:** @PrePersist runs before INSERT, @PostPersist after INSERT, @PreUpdate before UPDATE, @PostUpdate after UPDATE, @PreRemove before DELETE, @PostRemove after DELETE, @PostLoad after SELECT. Common use: auto-set createdAt in @PrePersist, updatedAt in @PreUpdate, audit logs in @PostPersist/@PostUpdate.

## 14.2 Complete JPA Annotations Cheat Sheet

| Category | Annotation | Purpose |
|---|---|---|
| Entity Basics | @Entity | Mark class as DB table |
| Entity Basics | @Table(name=) | Custom table name |
| Entity Basics | @Id | Primary key |
| Entity Basics | @GeneratedValue | Auto-generate PK |

| | | |
|---|---|---|
| Entity Basics | @Column | Column customization |
| Entity Basics | @Transient | Skip this field — not stored in DB |
| Relationships | @OneToOne | 1 to 1 relationship |
| Relationships | @OneToMany | 1 to many relationship (collection side) |
| Relationships | @ManyToOne | Many to 1 relationship (FK side) |
| Relationships | @ManyToMany | Many to many relationship |
| Relationships | @JoinColumn | Define FK column |
| Relationships | @JoinTable | Define join/bridge table for ManyToMany |
| Embedding | @Embeddable | Class fields embedded in parent table |
| Embedding | @Embedded | Use an embeddable class inside entity |
| Embedding | @AttributeOverride | Rename embedded columns |
| Inheritance | @Inheritance | Define inheritance strategy |
| Inheritance | @DiscriminatorColumn | Discriminator column for SINGLE_TABLE |
| Inheritance | @DiscriminatorValue | Value for each child in discriminator column |
| Inheritance | @MappedSuperclass | Shared fields parent — no table |
| Querying | @Query | Custom JPQL or native SQL query |
| Querying | @Param | Named parameter in @Query |
| Querying | @Modifying | Required for UPDATE/DELETE @Query |
| Querying | @NamedQuery | Pre-defined JPQL on entity class |
| Transaction | @Transactional | Run in DB transaction |
| Lifecycle | @PrePersist | Before INSERT |
| Lifecycle | @PostPersist | After INSERT |
| Lifecycle | @PreUpdate | Before UPDATE |
| Lifecycle | @PostUpdate | After UPDATE |
| Lifecycle | @PreRemove | Before DELETE |
| Lifecycle | @PostLoad | After SELECT (load) |
| Auditing | @CreatedDate | Auto-set creation timestamp |
| Auditing | @LastModifiedDate | Auto-set last update timestamp |
| Auditing | @CreatedBy | Auto-set creator username |
| Auditing | @LastModifiedBy | Auto-set last modifier username |
| Locking | @Version | Enable optimistic locking |
| Locking | @Lock | Set lock mode (pessimistic) |

## 14.3 Student Management System — Complete Entity Summary

```
// Student Management System — Complete Entity Map
```

```
// Summary of all entities in Student Management System

Student      <-- @Entity, has all the main student data
  |-- id (PK, @GeneratedValue IDENTITY)
  |-- name, email, phone, enrollmentYear, active
  |-- address (@Embedded <-- Address class with @Embeddable)
  |-- libraryCard (@OneToOne, owns FK library_card_id)
  |-- department (@ManyToOne, owns FK dept_id)
  |-- courses (@ManyToMany, owns student_course join table)
  |-- @PrePersist: sets createdAt, rollNumber

Department   <-- @Entity
  |-- id (PK), name, hodName
  |-- students (@OneToMany mappedBy='department', cascade=ALL, orphanRemoval=true)

LibraryCard  <-- @Entity
  |-- id (PK), cardNumber (unique), issueDate, expiryDate, active
  |-- student (@OneToOne mappedBy='libraryCard')

Course       <-- @Entity
  |-- id (PK), title, credits, instructor
  |-- students (@ManyToMany mappedBy='courses')

Address      <-- @Embeddable (NO table, NO @Id)
  |-- street, city, state, pincode
  |-- stored inside student table

// DB Tables created by JPA:
// student, department, library_card, course, student_course (join table)
```

## Final Interview Tip

When asked about JPA in an interview, always structure your answer like this:

- 1. Define JPA in one sentence: 'JPA is a specification for mapping Java objects to DB tables.'
- 2. Mention Hibernate: 'Hibernate is the most popular JPA implementation.'
- 3. Mention Spring Data JPA: 'Spring Data JPA builds on top of Hibernate and gives ready-made CRUD repositories.'
- 4. Give the Student Management System example: mention the entities, relationships, and queries.
- 5. Show advanced knowledge: mention N+1 problem, lazy loading, transactions, and @MappedSuperclass.

*Covering these 5 points shows the interviewer that you understand JPA deeply, not just superficially.*